Database& SQL

Prepared with AI Aid.

Setup — Installing Libraries & Creating Database Connection

Prerequisites & Installation

```
# Core data libraries
pip install pandas numpy

# Database access
pip install sqlalchemy

# APIs & web
pip install requests python-dotenv
pip install beautifulsoup4 lxml

# File formats
pip install openpyxl xlrd

# Recommended IDE
pip install jupyter ipython
```

Python Imports for This Tutorial

```
import sqlite3
import pandas as pd
from datetime import datetime, timedelta
import random

# For production (Section 1.5)
from sqlalchemy import create_engine, text

# Creating the connection
conn = sqlite3.connect('library.db')
cursor = conn.cursor()

# pd.read_sql_query returns a DataFrame
# sqlite3.connect returns a Connection object
# cursor enables raw SQL execution
```

Typical Workflow:

Connect to DB → Execute SQL → Fetch Results → Process with pandas → Close connection

CREATE TABLE — Syntax, Data Types & Constraints

```
cursor.execute('''
  CREATE TABLE IF NOT EXISTS books (
      book_id   INTEGER  PRIMARY KEY AUTOINCREMENT,  -- Unique auto-incrementing ID
      title     TEXT     NOT NULL,                   -- Cannot be empty
      author_id INTEGER  NOT NULL,                   -- Must reference an author
      isbn      TEXT     UNIQUE,                     -- No two books share an ISBN
      pub_year  INTEGER,                             -- Optional field
      genre     TEXT,
      copies    INTEGER  DEFAULT 1 CHECK(copies >= 0),-- Must be 0 or positive
      FOREIGN KEY (author_id) REFERENCES authors(author_id)  -- Relational link
  )
''')
```

Constraint Reference:

`PRIMARY KEY`	Unique row identifier
`AUTOINCREMENT`	Auto-assigns next integer
`NOT NULL`	Value required, no empty
`UNIQUE`	No duplicate values
`DEFAULT x`	Use x if no value given
`CHECK(cond)`	Row rejected if cond=False
`FOREIGN KEY`	Links to another table PK
`IF NOT EXISTS`	Skip if table already there

✅ Best Practice: Always use CREATE TABLE IF NOT EXISTS in scripts — prevents errors when re-running. Use explicit column names in INSERT (never rely on column order).

INSERT Data — Single Row, Multiple Rows & executemany()

Inserting data into tables using cursor.execute() and cursor.executemany()

```
# Method 1: Single row insert
cursor.execute(
    "INSERT INTO authors (name, email, country, birth_year) VALUES
(?, ?, ?, ?)",
    ('Naguib Mahfouz', 'mahfouz@literature.eg', 'Egypt', 1911)
)

# Method 2: executemany — batch insert (much faster for multiple
rows)
authors_data = [
    ('Naguib Mahfouz', 'mahfouz@literature.eg', 'Egypt', 1911),
    ('Taha Hussein',   'taha@literature.eg',    'Egypt', 1889),
    ('Nawal El Saadawi','nawal@literature.eg',  'Egypt', 1931),
]
cursor.executemany(
    "INSERT INTO authors (name, email, country, birth_year) VALUES
(?, ?, ?, ?)",
    authors_data     # Iterates over list automatically
)

conn.commit()        # IMPORTANT: Saves changes to disk permanently
```

Function Breakdown:

`cursor.execute(sql, params)`

Runs ONE SQL statement. params is a tuple of values that replace ? placeholders.

`cursor.executemany(sql, seq)`

Runs the same SQL for each item in seq. Much faster than looping execute().

`? placeholders`

Safely bind values (prevents SQL injection). Never use f-strings for user input!

`conn.commit()`

Saves all pending changes to disk. Without this, data exists only in memory.

SQL Level 1 — Basic SELECT Queries

SELECT — Retrieve data from tables

```python
# All columns, first 5 rows
query = "SELECT * FROM books LIMIT 5"
df = pd.read_sql_query(query, conn)

# Specific columns only
query = "SELECT title, genre, publication_year FROM books"
df = pd.read_sql_query(query, conn)

# Calculated column with alias
query = """
SELECT
    title,
    copies_available,
    copies_available * 50 AS inventory_value_egp
FROM books
"""
df = pd.read_sql_query(query, conn)
```

Sample Output:

```
   book_id    title        copies  value_egp
0        1  Cairo Trilogy     3       150
1        2  The Days          2       100
2        3  Woman at Zero     2       100
3        4  Yacoubian Bldg    4       200
```

Clause Reference:

`SELECT *`	All columns (avoid in production)
`SELECT col1, col2`	Specific columns only
`AS alias`	Rename column in output
`LIMIT n`	Return at most n rows
`col * 50`	Arithmetic on column values

SQL Level 2 — Filtering with WHERE, AND, OR, LIKE, IN, NULL

```sql
-- Single condition
WHERE author_id = 1

-- Multiple conditions (AND / OR)
WHERE genre = 'Fiction'
  AND publication_year > 1950
  AND copies_available > 1

-- Pattern matching (LIKE)
WHERE email LIKE '%@university.edu'
-- % = any number of chars
-- _ = exactly one char

-- Multiple values (IN)
WHERE membership_type IN ('student', 'faculty')

-- NULL handling (IS NULL / IS NOT NULL)
WHERE return_date IS NULL     -- Not yet returned
WHERE return_date IS NOT NULL -- Already returned
```

LIKE Pattern Cheatsheet

`LIKE 'A%'`	Starts with A
`LIKE '%@gmail.com'`	Ends with @gmail.com
`LIKE '%Cairo%'`	Contains word Cairo
`LIKE 'M_h_fouz'`	M + any char + h + any char + fouz

SQL Level 3 — Aggregation Functions

Aggregation functions collapse multiple rows into a single summary value.

Function	What it does	Example Result
`COUNT(*)`	Count ALL rows including NULLs	`COUNT(*) = 20 (all borrowings)`
`COUNT(col)`	Count rows where col is NOT NULL	`COUNT(return_date) = 14 (returned)`
`SUM(col)`	Total of numeric column	`SUM(fine_amount) = 84.0`
`AVG(col)`	Arithmetic mean (ignores NULLs)	`AVG(fine_amount) = 4.2`
`MIN(col)`	Smallest value	`MIN(publication_year) = 1929`
`MAX(col)`	Largest value	`MAX(fine_amount) = 12.0`

SQL Level 3 — GROUP BY & HAVING

GROUP BY groups rows with the same value and lets you aggregate within each group.

```
# Books grouped by genre with stats
query = """
SELECT
    genre,
    COUNT(*) as book_count,
    AVG(publication_year) as avg_year,
    SUM(copies_available) as total_copies
FROM books
GROUP BY genre
ORDER BY book_count DESC
"""
df = pd.read_sql_query(query, conn)
# Output:
#      genre  book_count  avg_year  total_copies
#    Fiction           7    1968.9            17
#  Biography           1    1929.0             2
```

📌 WHERE vs HAVING:

- WHERE filters individual rows BEFORE grouping
- HAVING filters groups AFTER GROUP BY aggregation

Rule: If using an aggregate function (COUNT, SUM…) in your filter → use HAVING

How GROUP BY Works:

```
BEFORE GROUP BY:
book_id  genre
1        Fiction
2        Biography
3        Fiction      ──▶  GROUP BY genre ──▶
4        Fiction
5        Fiction
6        Fiction
7        Fiction
8        Fiction

AFTER GROUP BY:
genre       COUNT(*)
Fiction     7
Biography   1
```

HAVING — Filter Groups (not rows):

```
SELECT genre, COUNT(*) as cnt
FROM books
WHERE publication_year > 1950
GROUP BY genre
HAVING cnt > 2
```

SQL Level 4 — JOINs: Connecting Tables

JOINs combine rows from two or more tables based on a related column (usually PK → FK).

authors table

```
author_id | name
1         | Mahfouz
2         | Hussein
3         | Saadawi
4         | No Books
```

books table

```
book_id | title          | author_id
1       | Cairo Trilogy  | 1
2       | The Days       | 2
3       | Woman at Zero  | 3
4       | Palace Walk    | 1
```

INNER JOIN result (author_id matches only):

```
author_id | name     | book_id | title
1         | Mahfouz  | 1       | Cairo Trilogy
1         | Mahfouz  | 4       | Palace Walk
2         | Hussein  | 2       | The Days
3         | Saadawi  | 3       | Woman at Zero
-- Author 4 "No Books" NOT included
```

Syntax:

```
FROM books b INNER JOIN authors a ON b.author_id = a.author_id
```

INNER JOIN

Only rows matching in BOTH tables. Author 4 ("No Books") excluded.

LEFT JOIN

ALL rows from left table + matching right rows. Author 4 shown with NULL book fields.

RIGHT JOIN

ALL rows from right table + matching left rows. (Less common in SQLite)

FULL OUTER JOIN

ALL rows from both tables. SQLite: simulate with UNION of LEFT+RIGHT JOINs.

SQL Level 4 — INNER JOIN: Full Code Example

```
# Books with their author names (INNER JOIN)
query = """
SELECT
    b.title,
    b.publication_year,
    a.name   AS author_name,    -- alias 'a' for authors table
    a.country
FROM books b                    -- alias 'b' for books table
INNER JOIN authors a            -- join authors
    ON b.author_id = a.author_id -- match condition
ORDER BY b.publication_year
"""
df = pd.read_sql_query(query, conn)
print(df)
```

```
Output:
           title  publication_year  author_name  country
0       The Days              1929 Taha Hussein    Egypt
1  Cairo Trilogy              1956Naguib Mahfouz  Egypt
2   Palace Walk              1956 Naguib Mahfouz  Egypt
3 Children Gebelawi           1959 Naguib Mahfouz  Egypt
4 God Dies by Nile            1974 Nawal Saadawi  Egypt
```

⚠️ Common Pitfall: If two tables have a column with the same name (e.g., both have "name"), always prefix with the table alias: a.name vs. m.name

SQL Level 4 — Three-Table JOIN & the CASE Statement

```
query = """
SELECT
    m.name          AS borrower_name,
    m.membership_type,
    bk.title        AS book_title,
    a.name          AS author_name,
    br.borrow_date, br.due_date, br.return_date,
    CASE
        WHEN br.return_date IS NULL             THEN 'Still Borrowed'
        WHEN br.return_date <= br.due_date      THEN 'Returned On
Time'
        ELSE                                         'Returned Late'
    END AS status,
    br.fine_amount
FROM borrowings br
INNER JOIN members m  ON br.member_id = m.member_id
INNER JOIN books   bk ON br.book_id   = bk.book_id
INNER JOIN authors a  ON bk.author_id = a.author_id
WHERE br.fine_amount > 0
ORDER BY br.fine_amount DESC
"""
```

CASE Statement — SQL's if/elif/else:

```
CASE
  WHEN condition1 THEN result1
  WHEN condition2 THEN result2
  ELSE             default_val
END AS column_name

-- Use anywhere in SELECT
-- Evaluated top-to-bottom
-- First matching WHEN wins
```

Multi-JOIN Pattern:

```
FROM main_table t1
JOIN table2 t2 ON t1.fk = t2.pk
JOIN table3 t3 ON t2.fk = t3.pk
JOIN table4 t4 ON t3.fk = t4.pk
-- Chain as many as needed
```

SQL Level 4 — LEFT JOIN: Finding Missing Relationships

LEFT JOIN returns ALL rows from the left table, and matching rows from the right (NULL if no match).

authors (LEFT)

```
author_id | name
1         | Mahfouz
2         | Hussein
3         | Saadawi
4         | No Books  ← kept!
```

books (RIGHT)

```
book_id | author_id | title
1       | 1         | Cairo Trilogy
2       | 2         | The Days
3       | 3         | Woman at Zero
```

LEFT JOIN result

```
name      | book_id | title
Mahfouz   | 1       | Cairo Trilogy
Hussein   | 2       | The Days
Saadawi   | 3       | Woman at Zero
No Books  | NULL    | NULL   ← NULL!
```

```python
# Use case 1: Find authors WITH NO books
query = """
SELECT a.name, a.email, COUNT(b.book_id) as book_count
FROM authors a
LEFT JOIN books b ON a.author_id = b.author_id
GROUP BY a.author_id, a.name, a.email
HAVING book_count = 0         -- Filter: only those with no books
"""

# Use case 2: Members who have NEVER borrowed anything
query = """
SELECT m.name, m.membership_type, COUNT(br.borrow_id) as borrow_count
FROM members m
LEFT JOIN borrowings br ON m.member_id = br.member_id
GROUP BY m.member_id
ORDER BY borrow_count          -- 0 = never borrowed
"""
```

SQL Level 5 — Subqueries: Queries Within Queries

A subquery is a SELECT statement nested inside another SQL statement.

Subquery in WHERE clause

```
# Find books by Egyptian authors born before 1950
query = """
SELECT title, publication_year
FROM books
WHERE author_id IN (
    SELECT author_id          ← inner query runs FIRST
    FROM authors
    WHERE country = 'Egypt'
      AND birth_year < 1950
)
ORDER BY publication_year
"""

# How it executes:
# Step 1: inner SELECT → returns [1, 2, 3] (matching author_ids)
# Step 2: outer SELECT → WHERE author_id IN (1, 2, 3)
```

Subquery Types:

In WHERE

Filter based on results of another query. Use with IN, NOT IN, EXISTS, =.

In SELECT

Compute a value per row (scalar subquery). Must return exactly 1 value.

In FROM

Treat subquery result as a table (derived table). Must use alias.

SQL Level 5 — Subqueries: Queries Within Queries

A subquery is a SELECT statement nested inside another SQL statement.

Subquery in SELECT clause

```
query = """
SELECT
    title,
    publication_year,
    (SELECT AVG(publication_year) FROM books) as avg_year,
    publication_year - (SELECT AVG(publication_year) FROM books)
         as year_vs_avg
FROM books ORDER BY year_vs_avg DESC
"""
```

Subquery Types:

In WHERE

Filter based on results of another query. Use with IN, NOT IN, EXISTS, =.

In SELECT

Compute a value per row (scalar subquery). Must return exactly 1 value.

In FROM

Treat subquery result as a table (derived table). Must use alias.

SQL Level 5 — CTEs (Common Table Expressions): Cleaner SQL

A CTE (WITH clause) names a temporary result set that you can reference in the main query — like creating a named step.

❌ **Without CTE (messy, nested):**

```
SELECT outer.name, inner1.count, inner2.avg
FROM (
    SELECT name,
           (SELECT COUNT(*) FROM (
               SELECT * FROM books WHERE ...
           )) as count
    FROM authors
) outer
JOIN (SELECT ...) inner2 ON ...
```

✅ **With CTE (clean, readable):**

```
WITH
  author_stats AS (
      SELECT author_id, name,
             COUNT(b.book_id) AS book_count
      FROM authors a
      LEFT JOIN books b USING(author_id)
      GROUP BY author_id, name
  ),
  borrow_stats AS (
      SELECT b.author_id,
             COUNT(br.borrow_id) AS borrows
      FROM books b
      LEFT JOIN borrowings br USING(book_id)
      GROUP BY b.author_id
  )
SELECT ast.name, ast.book_count,
       COALESCE(bs.borrows, 0) AS popularity
FROM author_stats ast
LEFT JOIN borrow_stats bs USING(author_id)
ORDER BY popularity DESC
```

Why CTEs Win:

- ✅ Readable: Named steps like author_stats are self-documenting
- ✅ Reusable: Reference the same CTE multiple times
- ✅ Debuggable: Test each CTE independently
- ✅ Optimizable: Modern DBs optimize CTEs like subqueries

COALESCE(value, default): Returns first non-NULL value.
COALESCE(bs.borrows, 0) → if borrows is NULL, use 0

1.5 SQLAlchemy — Production Database Access

SQLAlchemy is the standard Python ORM/database toolkit — preferred over raw sqlite3 in production.

🔌 Multi-DB
SQLite, PostgreSQL, MySQL, Oracle — one API

🔒 Safe
Built-in SQL injection prevention

⚡ Pooling
Reuses connections — no overhead per query

🐼 pandas
pd.read_sql() works seamlessly with engines

```python
from sqlalchemy import create_engine, text
import pandas as pd

# Create engine — connection string format:
# sqlite:///path/to/db.db
# postgresql://user:pass@host/dbname
# mysql+pymysql://user:pass@host/dbname
engine = create_engine('sqlite:///library.db')

# Method 1: Context manager (auto-closes connection)
with engine.connect() as conn:
    query = text("SELECT * FROM books WHERE genre = :genre")
    df = pd.read_sql(query, conn, params={'genre': 'Fiction'})

# Method 2: Safe parameterized query
with engine.connect() as conn:
    query = text("""
        SELECT * FROM borrowings
        WHERE member_id = :mid AND fine_amount > :min_fine
    """)
    df = pd.read_sql(query, conn, params={'mid': 1, 'min_fine': 5.0})
```

```python
# Method 3: Write DataFrame to DB
new_books_df.to_sql(
    'books',
    engine,
    if_exists='append',  # 'replace','append','fail'
    index=False          # Don't write df index as
col
)

# Method 4: Chunked reading (large tables)
for chunk in pd.read_sql(
    "SELECT * FROM borrowings",
    engine,
    chunksize=1000    # Process 1000 rows at a time
):
    process(chunk)    # Avoids memory overflow
```

Security — SQL Injection Attack & Prevention

SQL Injection is one of the most common and dangerous security vulnerabilities in database applications.

❌ VULNERABLE — Never do this

```
user_input = "' OR '1'='1"           # Malicious input from attacker
query = f"SELECT * FROM members WHERE email = '{user_input}'"
# Becomes: SELECT * FROM members WHERE email = '' OR '1'='1'
# '1'='1' is always TRUE → returns ALL members!
cursor.execute(query)    # ← Attacker gets every user's data
```

✅ SAFE — Parameterized Queries

```
# SQLite: use ? placeholders
cursor.execute(
    "SELECT * FROM members WHERE email = ?",
    (user_input,)           # Tuple — sqlite3 escapes it safely
)

# SQLAlchemy: use :named placeholders
with engine.connect() as conn:
    query = text("SELECT * FROM members WHERE email = :email")
    df = pd.read_sql(query, conn, params={'email': user_input})
    # SQLAlchemy handles escaping — malicious input just becomes a literal string
```

SQL Essentials — ORDER BY, LIMIT, OFFSET & DISTINCT

```
# ORDER BY — sort results
query = """
SELECT title, publication_year, copies_available
FROM books
ORDER BY publication_year DESC,    -- Primary sort: newest first
         copies_available ASC      -- Secondary: fewest copies first
"""

# LIMIT — return first N rows only (good for previews)
query = "SELECT * FROM books ORDER BY publication_year DESC LIMIT 5"

# OFFSET — skip N rows (used for pagination)
# Page 1: LIMIT 10 OFFSET 0
# Page 2: LIMIT 10 OFFSET 10
query = """
SELECT * FROM books
ORDER BY book_id
LIMIT 10 OFFSET 20    -- Skip 20, show next 10 (page 3)
"""

# DISTINCT — remove duplicate values
query = "SELECT DISTINCT genre FROM books"
query = "SELECT DISTINCT country FROM authors"
```

💡 Pagination pattern: page_size=10; offset = (page_num - 1) * page_size. Always add ORDER BY when using LIMIT/OFFSET — without it, rows have no guaranteed order!

Clause Quick Reference:

`ORDER BY col ASC`	Sort ascending (A→Z, 1→9)
`ORDER BY col DESC`	Sort descending (Z→A, 9→1)
`ORDER BY c1, c2`	Multi-column sort priority
`LIMIT n`	Maximum n rows returned
`OFFSET n`	Skip first n rows
`DISTINCT`	Remove duplicate rows

Database Workflows — Writing & Reading with pandas

```python
import pandas as pd
from sqlalchemy import create_engine

engine = create_engine('sqlite:///library.db')

# ── READING ──────────────────────────────────────
# Read entire table
df = pd.read_sql("SELECT * FROM books", engine)

# Read with parameterized query (safe!)
from sqlalchemy import text
with engine.connect() as conn:
    q = text("SELECT * FROM books WHERE genre = :g")
    df = pd.read_sql(q, conn, params={'g': 'Fiction'})

# Chunked read for large tables
chunks = []
for chunk in pd.read_sql("SELECT * FROM borrowings",
                         engine, chunksize=1000):
    chunks.append(chunk.query('fine_amount > 0'))
df = pd.concat(chunks)
```

```python
# ── WRITING
──────────────────────────────────────────────

new_df = pd.DataFrame({
    'title': ['New Book'],
    'author_id': [1],
    'genre': ['Fiction'],
    'copies_available': [3]
})

# if_exists options:
# 'fail'    → raise error if table exists
# 'replace' → DROP and recreate table
# 'append'  → add rows to existing table
new_df.to_sql(
    'books',      # table name
    engine,
    if_exists='append',
    index=False   # don't write DataFrame
index
)
```

Common Workflow:

Raw CSV/API → Clean with pandas → Write to DB (.to_sql) → Query with SQL → Analyze DataFrame